

PySpark Cookbook для Microsoft Fabric

Золотые инженерные шаблоны (Cheat Sheet)

Практическое руководство дата-инженера: готовые конструкции кода для блокнотов Fabric

Принцип инженера: Не нужно учить наизусть сотни методов PySpark. В 95% реальных задач используются базовые типовые паттерны (чтение $\rightarrow$ очистка $\rightarrow$ объединение $\rightarrow$ агрегация $\rightarrow$ оптимизированная запись в Delta).

1. Базовые импорты (Всегда в первой ячейке блокнота)

```
from pyspark.sql.functions import (
    col, lit, when, coalesce, concat, concat_ws,
    to_date, to_timestamp, year, month, dayofmonth, date_format,
    round, sum, avg, count, min, max, row_number
)
from pyspark.sql.window import Window
```

2. Чтение данных (Data Ingestion)

```
# 1. Чтение Delta-таблицы из текущего или соседнего Lakehouse
df = spark.read.table("lh_bronze.dbo.publicholidays")

# 2. Чтение Parquet-файла из раздела Files/
df_parquet = spark.read.parquet("Files/sample_datasets/public_holidays.parquet")

# 3. Чтение CSV-файла с автоопределением схемы и заголовками
df_csv = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("Files/sample_datasets/city_safety_seattle.csv")
```

3. Очистка, приведение типов и работа с NULL

```
# Фильтрация строк и переименование колонок
df_clean = df \
    .filter(col("date") >= "2020-01-01") \
    .filter(col("holidayName").isNotNull()) \
    .withColumnRenamed("countryRegionCode", "CountryCode")

# Преобразование строки в строгую Дату и извлечение атрибутов времени
df_dates = df_clean \
    .withColumn("HolidayDate", to_date(col("date"))) \
    .withColumn("Year", year(col("HolidayDate"))) \
    .withColumn("DayName", date_format(col("HolidayDate"), "EEEE"))

# Замена NULL (coalesce) и условная логика (when / otherwise)
df_final = df_dates \
    .withColumn("IsPaid", coalesce(col("isPaidTimeOff"), lit(False))) \
    .withColumn("Priority",
        when(col("DayName").isin("Saturday", "Sunday"), "Weekend")
        .otherwise("Workday")
    )
```

4. Объединение таблиц (Joins)

```
# Внутреннее объединение (INNER JOIN) по общему ключу
df_joined = df_orders.join(df_customers, on="CustomerID", how="inner")

# Левое объединение (LEFT JOIN), если ключи называются по-разному в таблицах
df_enriched = df_sales.join(
    df_products,
    df_sales["ProdKey"] == df_products["ProductID"],
    how="left"
).drop(df_products["ProductID"]) # удаляем дублирующийся столбец
```

5. Агрегации и группировки (Group By & Aggregations)

```
# Сводка: количество праздников по странам и годам
df_summary = df_final.groupBy("CountryName", "Year") \
    .agg(
        count("HolidayName").alias("TotalHolidays"),
        sum(when(col("IsPaid") == True, 1).otherwise(0)).alias("PaidHolidays")
    ) \
    .orderBy(col("Year").desc(), col("TotalHolidays").desc())
```

6. Оконные функции (Window Functions)

```
# Ранжирование строк: поиск последнего события по каждому пользователю
window_spec = Window.partitionBy("CountryName").orderBy(col("HolidayDate").desc())

df_ranked = df_final \
    .withColumn("RankNum", row_number().over(window_spec)) \
    .filter(col("RankNum") == 1) # оставляем только самый последний праздник по стране
```

7. Запись в Delta Lake (Silver & Gold)

```
# Перезапись всей таблицы (Overwrite) – для измерений и справочников
df_final.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("lh_silver.dbo.dim_holidays")

# Добавление новых строк (Append) – для фактов и логов
df_new_records.write \
    .format("delta") \
    .mode("append") \
    .saveAsTable("lh_gold.dbo.fact_daily_events")
```

8. Обслуживание Delta Lake & Time Travel (%%sql)

```
-- 1. Сжатие файлов и кластеризация для максимальной скорости Direct Lake / SQL
OPTIMIZE lh_silver.dbo.dim_holidays ZORDER BY (HolidayDate);

-- 2. Просмотр истории версий и коммитов
DESCRIBE HISTORY lh_silver.dbo.dim_holidays;

-- 3. Путешествие во времени: чтение состояния таблицы на версию 0
SELECT * FROM lh_silver.dbo.dim_holidays VERSION AS OF 0;

-- 4. Очистка старых версий файлов (по умолчанию старше 7 дней)
VACUUM lh_silver.dbo.dim_holidays;
```

Совет для Fabric: Функция `V-Order` включена в Microsoft Fabric по умолчанию при каждой записи через `saveAsTable()`. Она автоматически упорядочивает данные так, чтобы дашборды Power BI в режиме Direct Lake открывались мгновенно.